PARKING DAM-PRO — Git, diagramas y programación Java

Módulo: Programación · **Curso:** 1.º DAM
Repositorio: el **mismo** `parking-dam` que creaste en la Actividad 1
Herramientas: IntelliJ IDEA, Java 21, Maven, Git y GitHub
Evaluación: Actividad 1 y Actividad 2, cada una sobre 10 puntos · **Entrega:** un solo repositorio GitHub

> **Repositorio único:** `parking-dam`. La Actividad 1 trabaja Git, GitHub, Markdown y diagramas; la Actividad 4 implementa y prueba el algoritmo con Java. Todo se explica en este único README de la raíz. **No crees otro repositorio ni otro README**.

Actividad 1 — Git, GitHub, Markdown y diagramas (10 puntos)

En tu cuenta de GitHub crea un repositorio publico llamado **`parking-dam`** con README inicial. Clonalo en `~/git/parking-dam`, comprueba `git status` y `git remote -v` y aprende a preparar cambios (`git add`), confirmarlos (`git commit -m`) y publicarlos (`git push origin main`). Crea en diagrams.net los seis diagramas del algoritmo, guarda el editable `diagramas/parking.drawio` y exporta las seis imagenes `01_` a `06_`. Documenta tu trabajo en este mismo README utilizando titulos, subtitulos, listas, enlaces e imagenes.

Entrega y nota independientes: en la Actividad 1 se evalua la demostracion individual de comandos, el diseño y la documentacion (10 puntos). La Actividad 2 conserva otra nota sobre 10 y continua en el **mismo** historial de GitHub. Tras cada avance verificado en la Actividad 1, realiza tambien `git add`, `git commit -m` y `git push origin main`.

---

1. Objetivos y enunciado de la Actividad 4

Crearás un programa de consola llamado **PARKING DAM-PRO** que pida el nombre de usuario, muestre un menú hasta elegir salir y permita registrar vehículos, calcular tarifas y mostrar un resumen de la sesión. La tarifa es de **1,50 € por hora**.

Debes trabajar con **Java 21**, `Scanner`, `String`, `int`, `float`, `do-while`, `while`, `for`, `switch` (o `if-else`), contadores, acumuladores y **métodos definidos por ti**. No necesitas arrays, listas ni clases adicionales. Puedes suponer que las entradas numéricas tienen formato entero correcto: aquí se evalúa la **validación del rango**, no los errores por introducir letras.

Opción 1: preguntar cuántos vehículos registrar (entero positivo); para cada vehículo, pedir matrícula y horas positivas. Incrementar el total de vehículos y sumar las horas registradas.
Opción 2: pedir horas positivas, calcular `horas × 1.50f`, incrementar el contador de cálculos y acumular el importe. No cambiar contadores ni horas de la opción 1.
Opción 3: mostrar nombre, total de vehículos, total de horas registradas, cantidad de cálculos de tarifa e importe acumulado.
Opción 4: mostrar despedida y salir. Una opción diferente mostrará un aviso y volverá a presentar el menú.

2. Recuperar y sincronizar TU repositorio

Abre Git Bash o Bash y entra en tu repositorio de la Actividad 1:

```
cd ~/git/parking-dam
git status
git remote -v
git branch --show-current
git fetch origin
git log --oneline HEAD..origin/main
git pull --ff-only origin main
git status
```

Si es **otro ordenador** y todavía no tienes `parking-dam`, clónalo en `~/git/` mediante:

```
git clone https://github.com/TU_USUARIO/parking-dam.git
```

Comprueba que conservas `README.md`, `diagramas/parking.drawio` y tus seis imágenes.

Explicación para tu documentación:

> «Antes de programar comprobé [qué comandos], observé [estado real] y confirmé que el repositorio remoto era [URL de tu cuenta]».

3. Crear un proyecto Maven dentro de `parking-dam`

1. Abre **IntelliJ IDEA** y selecciona **File → New → Project**.
2. Selecciona **Java**, **Maven**, **JDK 21**, nombre **`parking-java`**.
3. Sitúa el proyecto **dentro de la carpeta que ya clonaste**: su ruta final debe ser `~/git/parking-dam/parking-java`.
4. Si aparece **Create Git repository**, **desactívalo**.
5. Conserva el `pom.xml` generado.
6. En IntelliJ, crea la clase Java **`Main`** con **New → Java Class** (como paso inicial del IDE).
7. **Renómbrala desde el IDE**, usando **Refactor → Rename** (`Shift + F6`), a **`Parking`**. Comprueba que la clase se llama `Parking`, que el archivo se llama `Parking.java` y que el método de entrada sigue siendo `public static void main(String[] args)`. No cambies solo el nombre del archivo desde el explorador.
8. Ejecuta `Parking.main()` y comprueba que la clase arranca antes de seguir.
9. **No crees `parking-java/README.md`**: toda la documentación se añade al `README.md` de la raíz `parking-dam`.

Estructura final esperada:

```
parking-dam/
├── README.md
├── .gitignore
├── diagramas/
│   ├── parking.drawio
│   ├── 01_inicio.png
│   ├── 02_menu.png
│   ├── 03_registro.png
│   ├── 04_tarifa.png
│   ├── 05_resumen.png
│   ├── 06_salida.png
│   └── 07_funciones_y_datos.png
└── parking-java/
    ├── pom.xml
    └── src/
        └── main/
            └── java/
                └── Parking.java
```

3.1. Evitar archivos generados en Git

En la raíz de `parking-dam`, crea o completa `.gitignore`:

```
**/target/
*.class
.idea/
*.iml
```

3.2. Primer commit de la Actividad 2

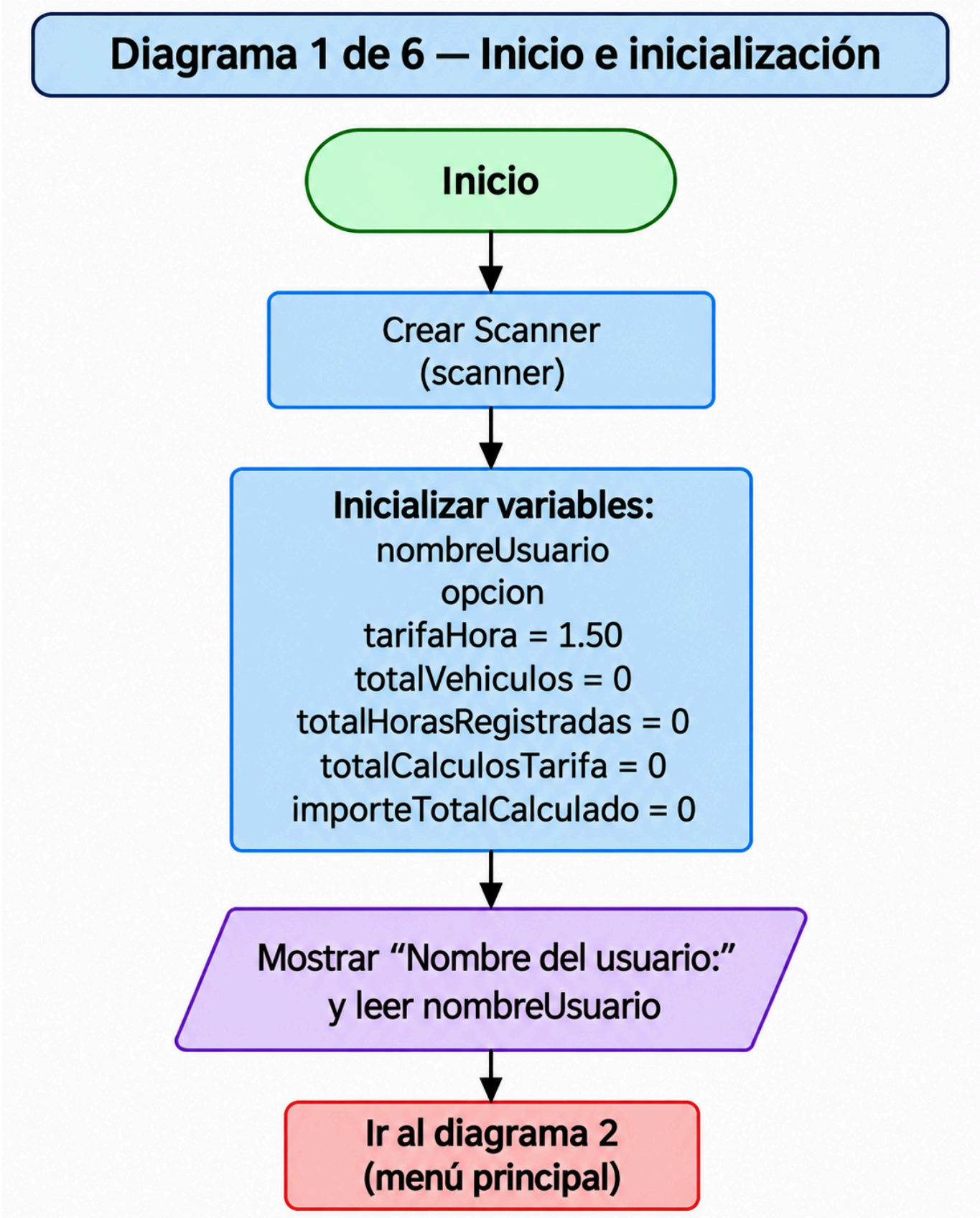
```
git status
git add .gitignore parking-java/pom.xml parking-java/src/ README.md
git diff --cached --stat
git commit -m "build: crear proyecto Maven parking-java"
git push origin main
```

---

4. Relación entre diagramas y código

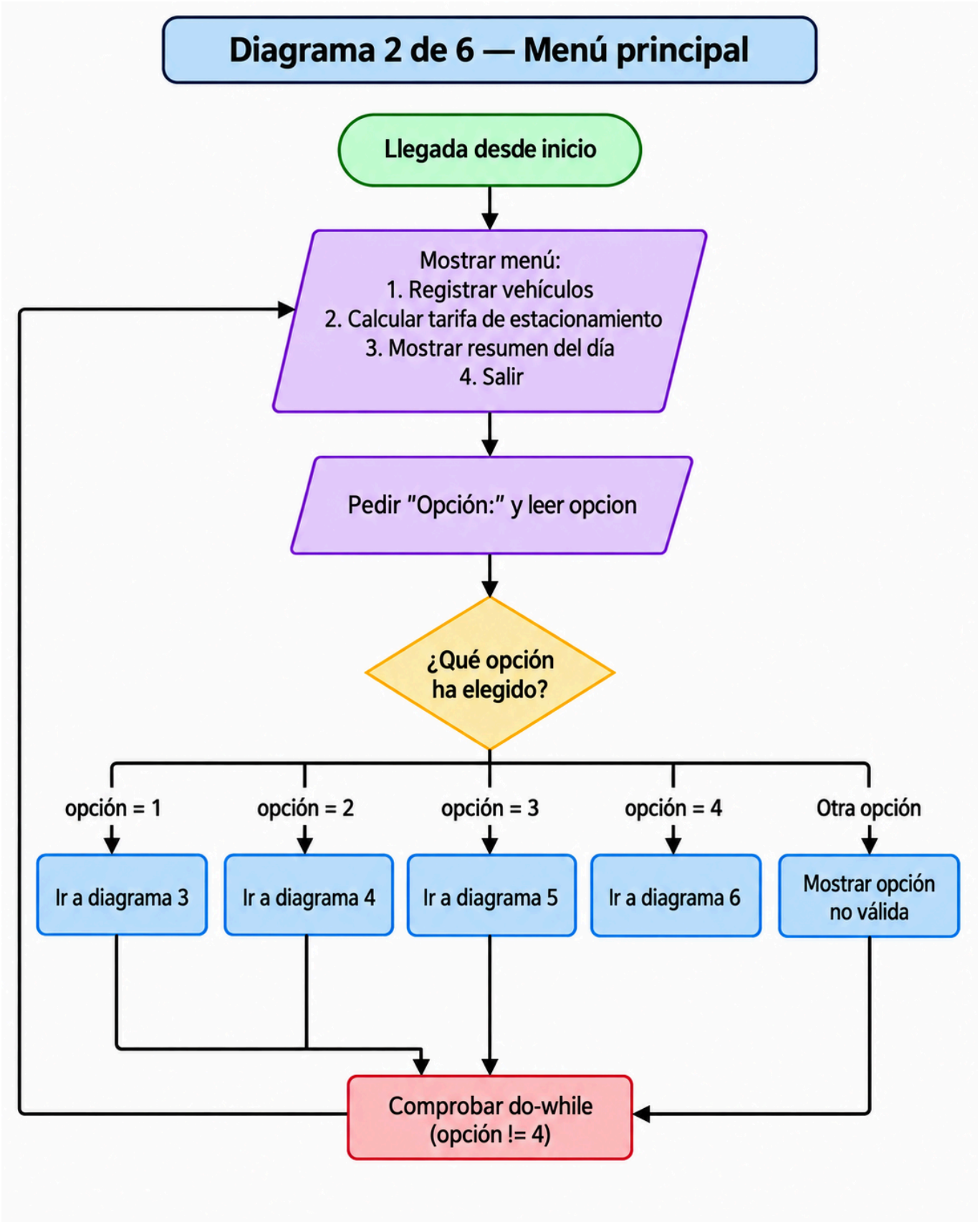
En el **único** `README.md` **de la raíz** documenta la relación entre diagramas y código. Las imágenes siguientes están integradas mediante rutas relativas a `diagramas/`. Explica debajo de cada una cómo se convierte en Java.

4.1. Diagrama 1 — Inicio e inicialización



Este diagrama representa el arranque del programa: creación de `Scanner`, inicialización de variables y lectura del nombre del usuario. En Java se corresponde principalmente con el comienzo de `main()`.

4.2. Diagrama 2 — Menú principal

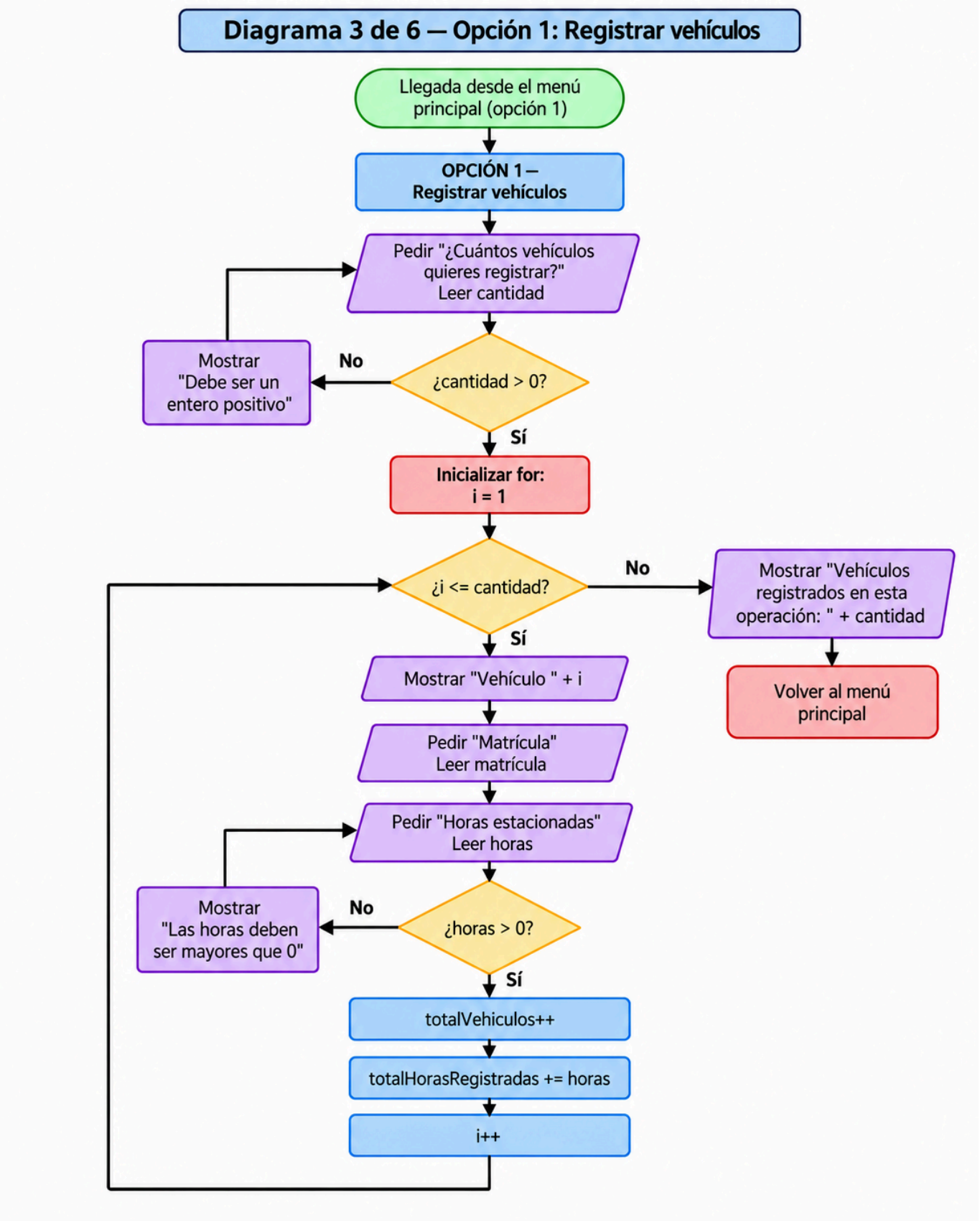


Este diagrama representa el menú, la lectura de la opción y la selección del flujo. Debes identificar que:

- las opciones **1, 2 y 3** vuelven a la comprobación del `do-while`,
- una **opción inválida** muestra un aviso y vuelve al menú,
- la **opción 4** conduce a la salida y **no** repite el menú.

En Java esto se resuelve con `mostrarMenu()`, un `switch` y un `do-while` en `main()`.

4.3. Diagrama 3 — Opción 1: Registrar vehículos



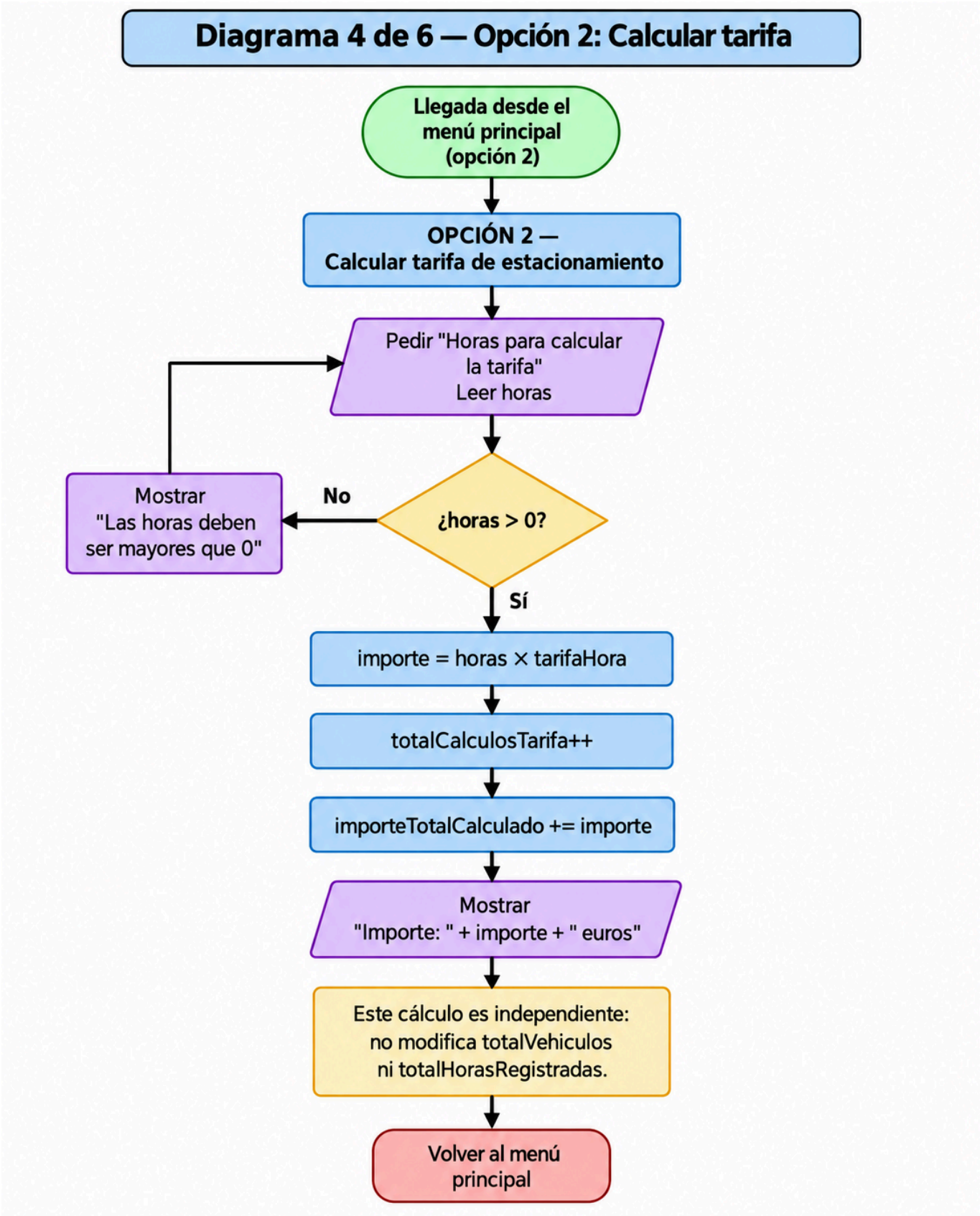
Este diagrama representa la validación de la cantidad, el inicio del `for`, la condición `i <= cantidad`, la petición de matrícula y horas, la validación de horas y la actualización de acumuladores.

Debes explicar en tu README que:

- primero se valida `cantidad > 0`,

- después se inicia el bucle con `i = 1`,
- el `for` continúa **solo mientras** `i <= cantidad`,
- cuando la condición es falsa, se muestra el total registrado y se vuelve al menú.

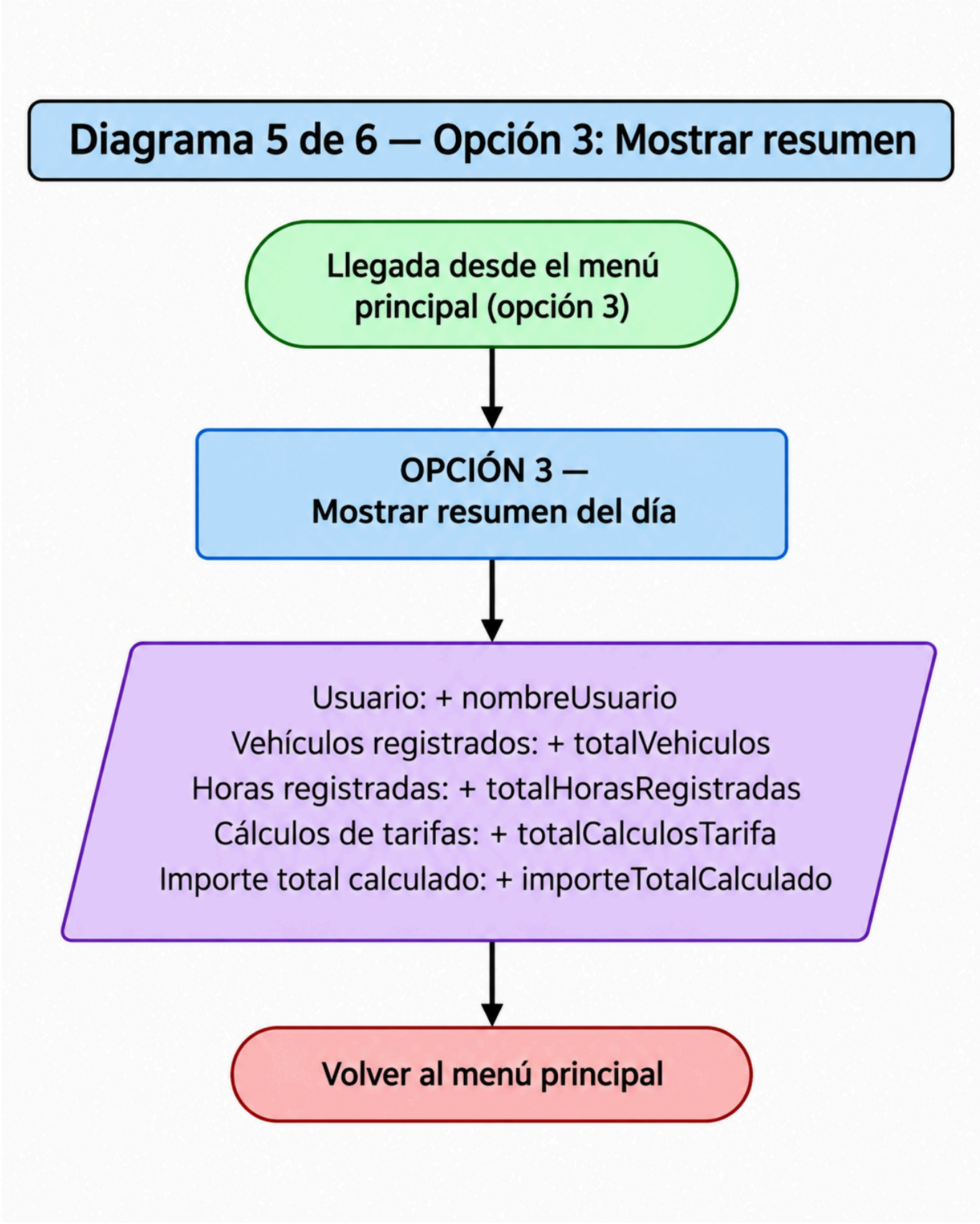
4.4. Diagrama 4 — Opción 2: Calcular tarifa



Este diagrama muestra la validación de horas positivas y el cálculo `horas × tarifaHora`. También debes reconocer que la opción 2:

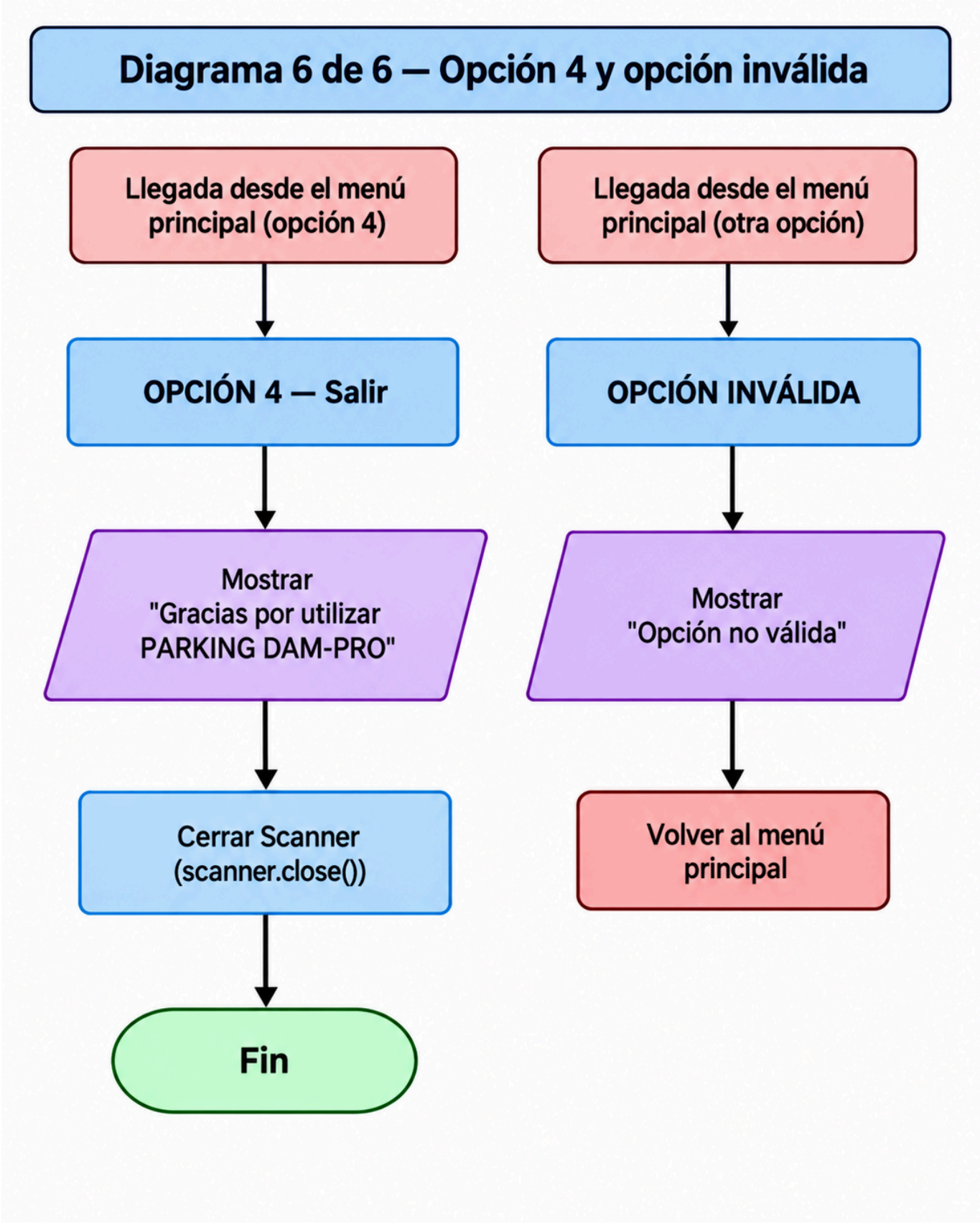
- incrementa `totalCalculosTarifa`,
- suma el importe a `importeTotalCalculado`,
- **no modifica** `totalVehiculos` ni `totalHorasRegistradas`.

4.5. Diagrama 5 — Opción 3: Mostrar resumen



Este diagrama corresponde al método `mostrarResumen(...)`. Debe enseñar al alumnado a mostrar los datos acumulados de la sesión: usuario, vehículos, horas, cálculos de tarifa e importe total.

4.6. Diagrama 6 — Opción 4 y opción inválida



Este diagrama representa dos casos distintos:

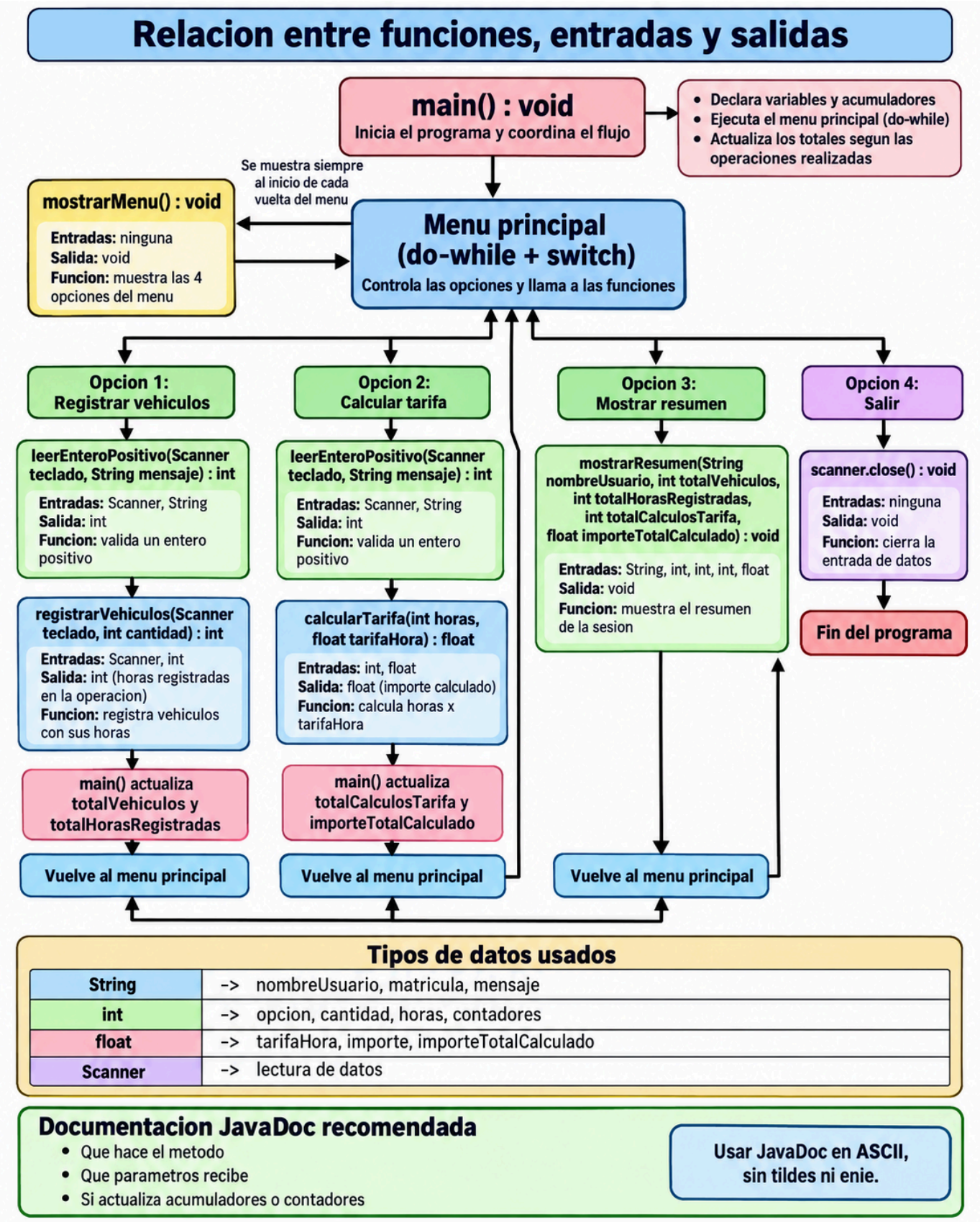
- **Opción 4:** mostrar despedida, cerrar `Scanner` y finalizar.
- **Opción inválida:** mostrar aviso y volver al menú.

---

5. Relación entre funciones, entradas y salidas

Además de los diagramas de flujo, debes entender **cómo se relacionan los métodos del programa** y qué tipo de datos reciben y devuelven.

5.1. Imagen-resumen de funciones y tipos de datos



Esta imagen resume la estructura del programa:

- `main()` coordina todo.

- `mostrarMenu()` muestra las opciones y no devuelve valor.
- `leerEnteroPositivo(...)` recibe un `Scanner` y un `String`, y devuelve un `int` validado.
- `registrarVehiculos(...)` organiza el registro con `for` y devuelve o produce los datos necesarios para actualizar acumuladores.
- `calcularTarifa(int horas, float tarifaHora)` devuelve un `float`.
- `mostrarResumen(...)` recibe los datos acumulados y no devuelve valor.
- `scanner.close()` cierra la entrada al finalizar.

5.2. Tabla de funciones, entradas y salidas

Método	Entradas	Tipo de salida	Qué hace
`main(String[] args)`	`String[] args` (argumentos del programa)	`void`	Coordina todo el programa
`mostrarMenu()`	ninguna	`void`	Muestra las 4 opciones
`leerEnteroPositivo(Scanner teclado, String mensaje)`	`Scanner`, `String`	`int`	Pide y valida un entero positivo
`registrarVehiculos(...)`	datos necesarios para el registro	según tu diseño	Registra vehículos y contribuye a actualizar acumuladores
`calcularTarifa(int horas, float tarifaHora)`	`int`, `float`	`float`	Calcula el importe de una tarifa
`mostrarResumen(...)`	`String`, `int`, `int`, `int`, `float`	`void`	Muestra el resumen de la sesión

5.3. Tipos de datos usados

Tipo de dato	Ejemplos dentro del programa
`String`	`nombreUsuario`, `matricula`, `mensaje`
`int`	`opcion`, `cantidad`, `horas`, `totalVehiculos`, `totalHorasRegistradas`, `totalCalculosTarifa`
`float`	`tarifaHora`, `importe`, `importeTotalCalculado`
`Scanner`	lectura de datos introducidos por teclado

5.4. Cómo documentar las funciones

En el único README de la raíz y en el código debes explicar:

- **qué hace** cada método,
- **qué parámetros recibe**,

- **qué valor devuelve**,
- y, si procede, **qué variables actualiza**.

Obligatorio: documenta **todos los métodos mediante JavaDoc**, con `/** ... */`, `@param` para cada parámetro y `@return` cuando corresponda. Los comentarios JavaDoc deben utilizar exclusivamente **caracteres ASCII** (sin tildes ni `ñ`).

Ejemplo de JavaDoc obligatorio

```
/**
 * Solicita un numero entero positivo y repite la lectura mientras el valor
sea menor o igual que cero.
 * @param teclado objeto Scanner para leer desde teclado
 * @param mensaje texto que se muestra antes de pedir el valor
 * @return entero positivo validado
 */
public static int leerEnteroPositivo(Scanner teclado, String mensaje) {
    // El alumno implementa aqui la lectura y la validacion.
    return 1; // Marcador provisional, no es la solucion.
}
```

Texto orientativo para el alumnado en su README:

> «He creado varios métodos para repartir responsabilidades. `main()` coordina el programa; `mostrarMenu()` presenta las opciones; `leerEnteroPositivo(...)` valida cantidades y horas; `calcularTarifa(...)` devuelve un `float` con el importe; y `mostrarResumen(...)` presenta la información acumulada. He documentado cada método indicando qué hace, qué datos recibe y qué valor devuelve».

5.5. Firmas recomendadas y JavaDoc obligatorio

Cada metodo debe tener un comentario JavaDoc inmediatamente anterior. Los textos de **JavaDoc deben usar solo ASCII**, sin tildes ni `ñ`. Estas son las firmas recomendadas; los bloques son ejemplos de documentacion y **no forman por si solos una clase Java compilable**, salvo el ejemplo completo de `calcularTarifa()`.

```
/**
 * Coordina la ejecucion del programa de aparcamiento.
 *
 * @param args argumentos de la linea de comandos
 */
public static void main(String[] args)

/**
 * Muestra las opciones disponibles del menu.
 */
public static void mostrarMenu()
```

```java
/**
 * Solicita y valida un entero positivo.
 *
 * @param teclado lector para obtener datos del usuario
 * @param mensaje texto mostrado al solicitar el dato
 * @return numero entero positivo validado
 */
public static int leerEnteroPositivo(Scanner teclado, String mensaje)

// Documenta el resto de funciones además de la clase con @autor y @version
```

El metodo `registrarVehiculos(...)` devuelve las horas de la operacion, que `main()` puede usar para actualizar los acumuladores sin variables globales:

```java
int horasOperacion = registrarVehiculos(teclado, cantidad);
totalVehiculos += cantidad;
totalHorasRegistradas += horasOperacion;
```

Ejemplo completo de metodo con JavaDoc:

```java
/**
 * Documente la funcion
 */
public static float calcularTarifa(int horas, float tarifaHora) {
    return horas * tarifaHora;
}
```

---

6. Dividir el programa en métodos

`main` debe **coordinar el menú**; no debe contener todo el programa. Como mínimo, crea los métodos siguientes:

Método sugerido	Para qué sirve
`mostrarMenu()`	Mostrar las cuatro opciones de la aplicación
`leerEnteroPositivo(Scanner teclado, String mensaje)`	Pedir un entero y repetir con `while` mientras no sea positivo; devolverlo
`registrarVehiculos(...)`	Registrar una cantidad conocida de vehículos con `for`, sumar vehículos y horas
`calcularTarifa(int horas, float tarifaHora)`	Devolver `horas × tarifaHora` como `float`
`mostrarResumen(...)`	Mostrar nombre, contadores y acumuladores

Advertencia importante: Java pasa los argumentos primitivos **por valor**. Si modificas un `int` dentro de un método, ese cambio no se refleja automáticamente fuera del método. Debes diseñar tus devoluciones y actualizaciones de forma coherente.

6.1. Plantilla orientativa

```
import java.util.Scanner;

public class Parking {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        float tarifaHora = 1.50f;
        int totalVehiculos = 0;
        int totalHorasRegistradas = 0;
        int totalCalculosTarifa = 0;
        float importeTotalCalculado = 0f;

        // 1. Leer el nombre del usuario.
        // 2. Repetir el menú con do-while.
        // 3. Resolver cada opción mediante switch.
        // 4. Llamar a tus métodos y actualizar acumuladores.
        // 5. Cerrar Scanner al salir.

        teclado.close();
    }

    // Escribe aquí tus métodos.
}
```

---

7. Implementacion por fases y publicacion obligatoria

Regla obligatoria en CADA fase: primero escribe el codigo y guardalo; despues ejecuta `Parking.main()` y verifica el resultado esperado. **Solo cuando funcione**, realiza `git status`, `git add`, `git diff --cached`, `git commit -m` y `git push origin main`. No acumules todas las modificaciones para un unico commit. Si no hay cambios, no crees commits vacios.

Fase 0 — Crear y renombrar Main a Parking

Crea `Main` desde IntelliJ, renombrala con **Refactor → Rename** a `Parking` y ejecuta la clase. Comprueba que el nombre del archivo, de la clase y la configuracion de ejecucion son correctos. Publica este primer avance:

```
git status
git add parking-java/src/ parking-java/pom.xml README.md
git diff --cached
git commit -m "refactor: renombrar Main a Parking desde IntelliJ"
git push origin main
```

Fase A — Menú que se repite

Antes del commit, ejecuta `Parking.main()` y verifica: elegir 9 muestra un aviso y vuelve al menú; elegir 4 sale sin volver a mostrarlo. Documenta el resultado real el README.md principal **parking-dam/README.md**.

```
git status
git add parking-java/src/ README.md
git diff --cached
git commit -m "feat: implementar menu principal con do-while"
git push origin main
```

Fase B — Registro de vehículos con `for`

Antes del commit, ejecuta `Parking.main()` y verifica: registrar dos vehículos de 2 y 3 horas debe sumar **2 vehículos y 5 horas**. Documenta el resultado real el README.md principal **parking-dam/README.md**.

```
git status
git add parking-java/src/ README.md
git diff --cached
git commit -m "feat: registrar vehiculos con for y validar datos"
git push origin main
```

Fase C — Calcular tarifa con un método

Antes del commit, ejecuta `Parking.main()` y verifica: 2 horas = **3,00 €**; 5 horas = **7,50 €**. DDocumenta el resultado real el README.md principal **parking-dam/README.md**.

```
git status
git add parking-java/src/ README.md
git diff --cached
git commit -m "feat: calcular tarifas y acumular importes"
git push origin main
```

Fase D — Resumen y revisión de métodos

Antes del commit, ejecuta `Parking.main()` y verifica: registra vehiculos, calcula al menos una tarifa y comprueba con la opcion 3 que los contadores y acumuladores permanecen correctos. Documenta el resultado real el README.md principal **parking-dam/README.md**.

```
git status
git add parking-java/src/ README.md
git diff --cached
```

```
git commit -m "feat: mostrar resumen mediante metodos"
git push origin main
```

---

8. Pruebas obligatorias del programa

Realiza las **12 pruebas** y escribe el resultado real en tu README:

Nº	Qué tienes que probar	Resultado esperado
1	Escribir un nombre	El menú saluda o identifica correctamente al usuario
2	Elegir 9	Error y regreso al menú
3	Introducir 0 como cantidad de vehículos, después 2	Rechaza 0 y acepta 2
4	Registrar 2 vehículos de 2 y 3 horas	Vehículos = 2; horas registradas = 5
5	En registro, introducir horas -2, después 4	Repite la pregunta hasta un valor positivo
6	Calcular tarifa de 2 horas	3,00 €
7	Calcular tarifa de 5 horas	7,50 €
8	En tarifa, introducir horas 0 y luego 3	Rechaza 0; importe 4,50 €
9	Calcular dos tarifas positivas	Contador de cálculos = 2
10	Registrar vehículos y luego calcular una tarifa	La opción 2 no altera vehículos ni horas registradas
11	Elegir opción 3 después de varias operaciones	Muestra todos los acumulados correctos
12	Elegir opción 4	Despedida y fin sin repetir menú

---

9. Documentacion obligatoria en el unico README principal

Este documento **es el unico** `README.md`, ubicado en la raiz de `parking-dam`. Conserva sus instrucciones y completa progresivamente los apartados de trabajo personal. No crees un segundo README dentro de `parking-java`.

9.1. Sintaxis de Markdown

Elemento	Como escribirlo
Titulo principal	`# PARKING DAM-PRO`
Apartado	`## Metodos implementados`
Subapartado	`### calcularTarifa()`

Elemento	Como escribirlo
Lista	`- Un elemento`
Nombre de metodo	`` `calcularTarifa()` ``
Enlace	`[Codigo Java](parking-java/src/main/java/)`
Imagen	`![Menu](diagramas/02_menu.png)`

9.2. Trabajo personal que debes anadir aqui

Desarrollo y explicaciones personales: explica como has creado y renombrado la clase `Main` a `Parking`, como se relaciona cada una de las siete imagenes con tu programa y como intercambian datos tus metodos. Especifica la entrada, salida y finalidad de cada metodo. No basta con copiar la tabla orientativa: escribe las firmas **reales** que has implementado.

Registro de comprobaciones: por cada fase (0, A, B, C y D) documenta la fecha, lo que probaste, el resultado real y el mensaje/identificador de commit publicado en GitHub.

Pruebas: completa las 12 pruebas de la seccion 8 con datos introducidos y resultados observados. Indica si alguna prueba fallo y como lo corregiste.

Dificultades y soluciones: describe al menos una dificultad real al usar Git, renombrar con IntelliJ o implementar los metodos.

9.3. JavaDoc obligatorio: ejemplo completo con ASCII

Debes escribir JavaDoc **inmediatamente encima de cada metodo**. Usa `@param` para cada parametro y `@return` si el metodo no es `void`. El texto de JavaDoc no debe contener tildes ni `ñ`.

```
/**
 * Calcula el importe total de un estacionamiento.
 *
 * Multiplica el numero de horas por el precio de una hora.
 *
 * @param horas numero de horas de estacionamiento
 * @param tarifaHora precio por cada hora de estacionamiento
 * @return importe total calculado
 */
public static float calcularTarifa(int horas, float tarifaHora) {
    return horas * tarifaHora;
}
```

En `main`, este metodo se llama cuando el usuario elige la opcion 2. **Devuelve** un `float`; no actualiza por si mismo los contadores: `main` incrementa `totalCalculosTarifa` y suma el importe a `importeTotalCalculado`.

En el resto de metodos debes documentar: `mostrarMenu()` (sin `@param` ni `@return`), `leerEnteroPositivo(...)` (con ambos `@param` y `@return`), `registrarVehiculos(...)` (segun su firma real) y `mostrarResumen(...)` (con `@param`, pero sin `@return` al ser `void`).

10. Commit final y publicación

```
git status
git add README.md parking-java/src/ parking-java/pom.xml
git diff --cached --stat
git commit -m "docs: documentar metodos y pruebas del parking"
git push origin main
git status
git log --oneline -8
```

11. Entrega en Moodle

Publica el enlace **directo a la carpeta** del proyecto Java en tu propio repositorio:

```
https://github.com/TU_USUARIO/parking-dam/tree/main/parking-java
```

El profesor revisará el proyecto Maven, el código Java, las pruebas documentadas, el unico README principal, la referencia a los diagramas de la Actividad 1 y los commits progresivos.

12. Rúbrica de la Actividad 2 (10 puntos)

Criterio	Puntos
Reutilizar repositorio y estructura Maven Java 21 correctos	1,0
Menú, `do-while`, opciones válidas e inválidas	1,0
Registro de vehículos con `for`, contador y acumulador de horas	1,5
Validación de cantidades y horas mediante `while`	1,0
Tarifa correcta y acumuladores independientes	1,0
Métodos: reparto de responsabilidades, parámetros y valores devueltos	2,0
Resumen correcto y estado conservado en toda la sesión	0,5
Doce pruebas con resultados reales, JavaDoc ASCII y README unico	1,5
Commits progresivos, publicación y enlaces entre ambas actividades	0,5
TOTAL	**10,0**

13. Ampliación voluntaria

Sin sustituir los requisitos anteriores, puedes diferenciar tarifa de coche (**1,50 €/h**) y de moto (**0,75 €/h**). Si haces la ampliación, documéntala y publícala en un commit propio:

```
git commit -m "feat: aplicar tarifas por tipo de vehiculo"
```